Question	Answer
what if we need data from multiple tables in the same api call?	One API call can still hit multiple tables — you combine them in the service layer, usually with SQL.
Does it mean that if GraphQL is present then we don't need manual implementations of POST, GET request handler APIs in route.ts on server side	No. GraphQL does not remove the need for server-side handlers. It changes what you write, not whether you write them. GraphQL replaces many REST routes with one GraphQL endpoint + resolvers. You still write server logic; the shape is just different. If you're on Next.js route.ts, you'd typically have something like app/api/graphql/route.ts instead of ten separate REST files — not zero handlers.
does PRISMA works only with postgres? Do we have similar libraries for NoSQL	Prisma = mostly relational DBs. For NoSQL, use the official driver or an ODM like Mongoose for MongoDB. There isn't one universal Prisma equivalent across all NoSQL databases — each DB has its own ecosystem.
Can Prisma handle Stored Proc and user defined functions?	Prisma can call them, but not manage them well. Stored procedures No first-class support — you can't define or migrate stored procs in Prisma schema.
Do we use the ORM in Azure Cosmos DB for mapping?	Use the Cosmos SDK (or Mongoose for Mongo API)
If possible, could you also include GraphQL Federation and provide some references or resources to learn more about it?	GraphQL Federation lets multiple teams run separate GraphQL services (subgraphs) that compose into one unified API for clients.
As we are adding one more layer of prisma between backend and postgres, will it slow down the processing time ??	Prisma adds a thin layer of cost. For typical backends it's negligible compared to Postgres and network time. Optimize the DB (indexes, query shape) before worrying about removing Prisma.
then how do we handle a scenarion where we have a complexy query for say a report generation?	Prisma for day-to-day app data. For reports — SQL you control (raw, view, or function), often async, sometimes on a read replica so you don't slow down production writes.
Can we have the hands on GraphQL Federation with microservices like structure?	Yes — GraphQL Federation is built specifically for that: each microservice exposes its own GraphQL subgraph (its own schema and resolvers), and a gateway (like Apollo Router or GraphQL Mesh) stitches them into one unified graph for clients to query. If you want, share your service boundaries and I can sketch a concrete federation setup (subgraph schemas, entity keys, and gateway config) for hands-on implementation.
so this is not the consideration for a report genration scenario? rt?	Don't pick or drop Prisma for reports because of speed. Pick how you run the query (SQL, async, replica) based on report size and freshness.

Can Prism be configured for an existing DB with some data?	Yes. Prisma can be pointed at an existing database with data already in it by running `prisma db pull` (introspection) to generate a matching schema from your current tables, after which you configure the connection and use it as normal.
cant we do same with pydantic models?	No. Pydantic only validates and types Python objects — it does not connect to Postgres or read your DB schema. For the Prisma-like part in Python, use SQLAlchemy or SQLModel for DB access, and Pydantic for API request/response models.
or is there a tool exist to geerate a schema from the existing DB?	Yes. Prisma's own introspection command, `prisma db pull`, does exactly that by reading your existing database and generating a matching `schema.prisma` file automatically.
Hi, i just joined the meeting, please can we discuss what advnatage does prisma give us? i mean we are already using typescript for type safety and using fastapi in backend with explicitly declared fields, isn't this just an extra step? i dont understand	TypeScript and FastAPI types only protect your app code — not your database. Prisma adds: Talks to Postgres — connection pooling, queries, transactions (types alone can't do this) Schema ↔ code sync — if a column is removed in DB, Prisma types break at build time Migrations — version-controlled DB changes Query builder — type-safe where, include, select without hand-writing SQL for every CRUD call Without Prisma you'd still need something — raw SQL, pg, SQLAlchemy, etc. You'd manually keep types in sync with the DB, which drifts over time.
IS Prisma a open source ORM tool?	Prisma is open source (Apache 2.0). The core ORM, CLI, and client are free. Some extras — like Prisma Data Platform / Accelerate — are paid cloud services, but the ORM itself is open source.

Can we use or setup GraphQL with Prisma as type, model and schema also define in GrsphQl?	Yes, they work well together — but they serve different roles. Prisma defines your DB schema (schema.prisma) and generates a type-safe client for Postgres. GraphQL defines your API schema (what clients can query) — usually in .graphql files or code-first with Pothos, Nexus, or TypeGraphQL. Typical setup: Prisma talks to the DB; GraphQL resolvers call prisma.user.findMany() etc. You can generate GraphQL types from Prisma with tools like Pothos + prisma plugin or typegraphql-prisma, but you still maintain two layers — DB schema (Prisma) and API schema (GraphQL) — unless you use codegen to sync them.
can we get /Select specific columns in the result or it would give the complete db rows of the result as the object .	Yes, Prisma lets you select specific columns instead of getting the full row: by default a query like `findMany()` returns all scalar fields, but you can pass a `select` option (e.g. `prisma.user.findMany({ select: { id: true, email: true } })`) to fetch only the columns you want, or use `include` to also pull in related tables' data alongside chosen fields.
How is it different from Hibernate ORM tool?	Prisma is a TypeScript/JavaScript ORM that uses a separate declarative schema file to generate a fully type-safe client with explicit, stateless queries and built-in migrations, whereas Hibernate is a Java/JPA ORM that maps annotated entity classes directly to tables and manages them through a stateful, session-based persistence context with implicit dirty-checking and caching. In short, Prisma favors explicit, type-safe simplicity for the JS ecosystem, while Hibernate offers a more feature-rich but implicit, "enterprise" object-relational mapping model for the JVM ecosystem.
if using prisma , then we can write only simple crud queries, what if we want custom or complex queries?	You can still run complex queries — just not always through Prisma's builder. Simple CRUD → use Prisma's API (findMany, create, etc.) Complex queries → use $queryRaw or $executeRaw with your own SQL (JOINs, CTEs, aggregations, reports). Prisma runs the SQL and returns results — you keep the connection and pooling.

Can you please provide an example where prisma might also be prone to SQL Injection	Prisma can be vulnerable to SQL injection when using its raw query methods—like `$queryRawUnsafe` or `$executeRawUnsafe`—if user input is directly concatenated into the query string, e.g. ``` prisma.$queryRawUnsafe(`SELECT * FROM User WHERE email = '${userInput}'`) ```, since these methods bypass Prisma's automatic parameterization. This risk doesn't apply to the standard type-safe API (like `findMany` with `where` clauses) or to `$queryRaw` used properly with tagged template literals, both of which safely parameterize inputs by default.
Is it required to install PostgreSQL also?	Prisma doesn't include Postgres — you need a Postgres database somewhere.
Can we see the SQL generated by prisma , is there a chance the SQL generated by prisma for complex queries is not fully optimized and can be slow to run ???	Yes, you can see the SQL. Enable query logging in Prisma: const prisma = new PrismaClient({ log: ['query'] }); Or use Prisma Studio, or inspect logs in dev. You'll see the exact SQL sent to Postgres.
does prisma studio run on different port or same as postgresql server?	Different port. Prisma Studio is a local web UI (usually http://localhost:5555) that connects to your Postgres server (default port 5432). Studio doesn't replace or run on the same port as Postgres — it's a separate dev tool on your machine.
then again its query raw which is query right then sql injection ? then prisma? could u explain	Prisma protects you in both cases — if you use it correctly. Normal Prisma queries (findMany, create, etc.) — fully safe. Prisma uses parameterized queries; user input never goes into the SQL string. $queryRaw with tagged template — also safe: await prisma.$queryRaw`SELECT * FROM users WHERE id = ${userId}`; Prisma turns ${userId} into a bound parameter, not string concatenation. $queryRaw with plain string — unsafe: await prisma.$queryRawUnsafe(`SELECT * FROM users WHERE id = '${userId}'`);
will Prisma handle rollback automatically?	Not fully automatically on its own, Prisma only rolls back changes automatically within an explicit `$transaction` block if an error is thrown (all queries in that transaction are reverted), but individual queries run outside a transaction are not automatically rolled back if a later, separate query fails.

So can we say Prisma acts as a wrapper on top of any database ? How is it different from directly crud/jpa repository ?	Yes, Prisma essentially acts as a type-safe wrapper/abstraction layer that translates its client API calls into raw SQL for the underlying database (Postgres, MySQL, SQLite, etc.), whereas a JPA/CRUD repository (like Spring Data JPA) is itself built on top of a full ORM (Hibernate) that manages a stateful persistence context, entity lifecycle, and implicit dirty-checking—so Prisma is generally more lightweight and explicit, while JPA repositories offer more automatic, object-graph-aware persistence management at the cost of added complexity and "magic."
Can you also talk about the limitatios of Prisma if any?	Not universal — only supports specific DBs (Postgres, MySQL, SQLite, etc.), not Cosmos, Redis, etc. Complex SQL is awkward — heavy JOINs, CTEs, reports → you fall back to $queryRaw Stored procs / UDFs — callable but not managed in schema or migrations N+1 risk — careless include can fire many queries Generated SQL — may be slower than hand-tuned SQL on hot paths Serverless cold starts — client + engine add startup cost Two schemas to maintain — Prisma schema + GraphQL/API layer if you use both Large apps — some teams outgrow it and move hot paths to raw SQL or lighter tools (Drizzle, pg) MongoDB support — limited compared to Mongoose No full DB admin — migrations and Studio help, but not a replacement for DBA tooling
So Prisma can be used only when we are using Javascript/Typescript based backend like nodejs, nextjs etc, correct me if I am wrong?	Prisma is built for JavaScript/TypeScript (Node.js, Next.js, NestJS, etc.). That's where it's mature and widely used.
$queryRaw / $executeRaw = write custom SQL yourself for complex cases then why prisma? couldnt understand if using query raw sql injection ?isnt it?	ou use Prisma for 80% simple CRUD — type-safe, no SQL strings, migrations, schema sync. For the 20% complex cases (reports, CTEs), you drop to $queryRaw only where needed. You still get connection pooling, one client, and safe parameter binding — without writing all SQL yourself.
will this support cascade delete?	es. Prisma supports cascade delete in two ways: Database level — set onDelete: Cascade on a relation in schema.prisma; Prisma migrations apply it to Postgres foreign keys. Deleting a parent row auto-deletes children in the DB. Prisma Client — when you delete() a parent, related records are removed if the FK has ON DELETE CASCADE in the DB.

Can Prisma ORM be used with MySQL?	Yes. Prisma supports MySQL (and MariaDB). Set provider = "mysql" in schema.prisma and point DATABASE_URL at your MySQL instance. Same Prisma API — findMany, migrations, Studio — works the same as with Postgres.
for shared repo push we can also use liquibase for version control instead of using Prisma ?	Yes. Liquibase can be used instead of Prisma Migrate to version-control and manage database schema changes via changelogs in a shared repo, since it's a database-agnostic migration tool that works independently of any specific ORM, though you'd lose Prisma's tight integration where the schema file and generated type-safe client stay automatically in sync with migrations.
How can we find, update, or create a record if it doesn't exist, using a single Prisma query?	Use upsert — one Prisma call that updates if the record exists, creates if it doesn't: await prisma.user.upsert({ where: { email: "gaurav@example.com" }, update: { name: "Gaurav" }, create: { email: "gaurav@example.com", name: "Gaurav" }, }); where must use a unique field (id, email, etc.). Prisma translates this to a single INSERT ... ON CONFLICT (Postgres) or equivalent on MySQL.
why postgress is used not sql ?	Postgres is SQL — SQL is the language; PostgreSQL is a database that runs SQL (like MySQL, SQL Server, Oracle).

In prisma deploy, we should have a server hosted somewhere right? How does it know which prod server to deploy the schema to?	When you run prisma migrate deploy (in CI/CD or on deploy), it connects to whatever database that env var points to:
	
	# prod pipeline sets this
	DATABASE_URL="postgresql://user:pass@prod-db-host:5432/mydb"

ya why not use mysql or sqllite database	Prisma doesn't pick a server on its own — you tell it via DATABASE_URL.
	
	When you run prisma migrate deploy (in CI/CD or on deploy), it connects to whatever database that env var points to:
	
	# prod pipeline sets this
	DATABASE_URL="postgresql://user:pass@prod-db-host:5432/mydb"
	Typical setup:
	
	Dev → local Postgres URL in .env
	Staging → staging URL in staging CI/env
	Prod → prod URL in prod CI/env (secrets vault, GitHub Actions, Vercel, etc.)
	You need:
	
	A hosted Postgres somewhere (RDS, Supabase, Neon, etc.)
	Your app server deployed (Vercel, AWS, etc.)
	Migrate step in deploy pipeline with the correct prod DATABASE_URL
	Bottom line: Same Prisma command everywhere — the environment variable decides which DB gets the schema. Wrong URL = wrong DB, so prod secrets must be locked down in CI/CD.
	
	ya why not use mysql or sqllite database
	
	You can — Prisma supports MySQL and SQLite too. Postgres is a common default, not a requirement.

How does Prima know that what is changed in a DB for an upgrade scenario? any temp tables gets created in DB?	If someone changes prod DB manually (outside Prisma), Prisma won't know — your repo and DB drift apart. Fix with prisma migrate diff or db pull to reconcile.
Then what is the difference between migrate dev and deploy if the DB server is same for deploy and migrate dev?	If both point at the same DB: bad idea for prod — migrate dev can reset data and is meant for dev only. Best practice: migrate dev on local/dev DB; migrate deploy on staging/prod with the same migration files from git. dev = develop and generate migrations. deploy = run already-approved migrations safely in prod.

I use Rails for backend. In it, I have schema.rb file. If I use prisma, will the db be defined in schema.prisma or schema.rb or both?	Rails owns the DB via ActiveRecord: migrations in db/migrate/, snapshot in schema.rb. Prisma owns it via schema.prisma + prisma/migrations/. Prisma is for Node/TypeScript backends, not Ruby/Rails. A Rails app doesn't use Prisma — it uses ActiveRecord, which already gives you models, migrations, and schema.rb. If you somehow used both on the same DB: you'd have two sources of truth (schema.rb + schema.prisma) — they drift, migrations conflict. Don't do that.
Is prisma only used for nodejs or it support other frameworks	Prisma is primarily built for the Node.js/TypeScript ecosystem, but it also officially supports other JS/TS-based frameworks and runtimes built on top of it, like Next.js, NestJS, Express, Deno, Bun, and even edge/serverless environments via Prisma Accelerate, though it doesn't support non-JavaScript languages like Java, Python, or Go (those ecosystems have their own separate ORMs).
Like Prisma for relational database, do we have for non relational databases? or is it not needed or we can use Prisma itself?	Prisma works mainly with SQL databases (Postgres, MySQL, SQLite). It has limited MongoDB support, but not Redis, DynamoDB, Cosmos, etc.
OKTA & microsoft Authenticator works as authenticators. What makes Oauth different from those?	Okta / Microsoft Authenticator = who you are (identity + login) Okta = identity platform (SSO, user directory, MFA) Microsoft Authenticator = MFA app (approves login on your phone) OAuth = protocol for granting access (a standard, not an app) Lets an app access your data without your password Example: "Login with Google" or an API accessing your calendar
Is active recod of rails and prisma similar in what it provides?	ActiveRecord is inside Rails; Prisma is standalone for Node/TS. If you know Rails models, Prisma Client will feel familiar — same idea, different syntax and ecosystem.
is Prisma same as Entity frame work?	Both are ORMs — they map database tables to code objects and handle queries/migrations. EF Core is Prisma for the .NET world

How validation of access token works? Whether it's valid or not?	Your API checks the token on every request — the auth server issued it, your API verifies it. Common steps: Extract token — from Authorization: Bearer <token> header Check signature — JWT signed by auth server (Okta, Azure AD, etc.); verify with their public key (JWKS endpoint) Check claims — not expired (exp), correct audience (aud = your app), correct issuer (iss) Optional — check scopes/roles (read:orders), call introspection endpoint for opaque tokens
How can we use NextAuth.js to authenticate against Azure Active Directory while enforcing custom, app-specific roles or permissions?	NextAuth = Azure login. Roles live in Azure app roles, your DB, or group mapping — then inject into session and check on every protected route/API. NextAuth handles login via Azure AD; you manage roles yourself. Setup: Register app in Azure Entra ID (AD) → get Client ID, Client Secret, Tenant ID Configure NextAuth Azure AD provider with those credentials User logs in → NextAuth gets Azure token + profile
session:{strategy:database } ? what does this mean?	it means sessions are stored in your database, not in a cookie/JWT.
how prisma relates to spring boot rest data framework which also provides api layer if data objects are created	They operate at different layers — Prisma is not an API framework. Prisma replaces JPA, not Spring Boot. You still write API routes in Node; Spring Boot can generate those for you if you use Spring Data REST.
how issue JWT and when TLS security is used?	After the user proves identity (login via Okta, Azure AD, username/password), the auth server issues a JWT containing claims (user id, roles, expiry). The client stores it and sends it on later requests (Authorization: Bearer <token>). Your API validates it — no login call every time.
We can also generate JWT token using certain libraries for DB based authentication as well right?	After the user logs in with username/password (checked against your DB), your backend issues a JWT using a library — no Okta/Azure needed. User sends email + password → You verify against DB (Prisma/bcrypt) → If valid, sign a JWT with a secret key → Return token to client

Can NextAuth support custom auth? like verify username paswword with our own DB?	Yes. NextAuth supports custom DB login via the Credentials provider.
	CredentialsProvider({
	async authorize(credentials) {
	const user = await prisma.user.findUnique({ where: { email: credentials.email } });
	if (!user || !bcrypt.compareSync(credentials.password, user.passwordHash))
	return null;
	return { id: user.id, email: user.email };
	}
	}) NextAuth isn't only for Google/Azure/Okta — Credentials provider = your own DB auth. You handle hashing, validation, and roles; NextAuth handles session management.

i believe google will not allow localhost callback URL's , sow how will app registrations works	Authorized redirect URIs: http://localhost:3000/api/auth/callback/google ← dev https://myapp.com/api/auth/callback/google ← prod
What are the use cases for JWT-based sessions compared to database sessions? Should they ever be combined, or is choosing one enough?"	JWT sessions — token in cookie/header, no DB lookup per request. Fast, stateless, good for APIs and serverless. Hard to revoke instantly (valid until expiry). Database sessions — session ID in cookie, full session in DB. Easy to revoke (delete row), audit trail, good when you need "logout everywhere" or instant ban.
Is the Microsoft's or any other Authentication architecture similar as Google, or it changes as per the provider ?	Same core architecture — different provider details. All major providers (Google, Microsoft/Azure AD, Okta, Auth0) use OAuth 2.0 + OpenID Connect (OIDC). The flow is the same: App → redirects to provider login → user authenticates → provider redirects back with code → app exchanges code for tokens
Is it advisable to use both JWT and DB sessions, or is one sufficient?	One is usually enough — pick based on your needs. Use JWT if you want speed, serverless/API scale, and can live with delayed revoke (until token expires). Use DB sessions if you need instant logout, ban users immediately, or session audit trails.
should we write these code for each function? or can we enable for the entire app?	Enable it app-wide — don't repeat auth in every function.

how to do so in the app level?	Next.js / NextAuth: middleware.ts — protects all matching routes in one place export { default } from "next-auth/middleware"; export const config = { matcher: ["/dashboard/:path*", "/api/:path*"] }; API routes: wrap once with a helper or use middleware — not per-function if (!session) everywhere. Express/NestJS: use middleware or guards applied at router/app level. app.use("/api", authMiddleware); // all /api routes protected
Can we make the redireect url dynamic, the page where the user last visited or from page where the authentiction started ?	Yes — pass the original page as a callbackUrl and redirect there after login.
How JWT is related to OAuth2.0?	OAuth 2.0 is the protocol (how apps get permission to access resources). JWT is a token format (how identity/permissions are packaged). Relationship: OAuth defines the flow (login → get token → use token on API calls) JWT is one way to encode the access token or ID token — not required, but very common OAuth flow → issues tokens → token is often a JWT (signed JSON with user id, scopes, expiry)
JWT is one way of implementation to tokenize in OAuth 2.0 - is this correct thinking?	Yes — that's correct. OAuth 2.0 defines how you get a token (flows, grants, endpoints). It does not mandate JWT as the token format. Token can be: JWT — structured, self-contained, verifiable locally (most common today) Opaque — random string, validated by calling the auth server So: OAuth 2.0 = the protocol. JWT = one popular implementation choice for encoding the token. Your thinking is right.
what is best for an intranet application? an application inside a company? not exposed to outside?	For an internal company app, use your company's identity provider (SSO) — not DIY username/password.
is bcrypt part of NextAuth?	No. bcrypt is a separate npm package (bcrypt / bcryptjs), not part of NextAuth.

what is 12 salt rounds?	Salt rounds = how many times bcrypt hashes the password. Higher = slower to crack, slower to verify.

How can we use a unique salt for every user's password with bcrypt?	You don't need to do it manually — bcrypt does it automatically.
	
	Every call to bcrypt.hash() generates a unique random salt and embeds it in the hash string:
	
	const hash = await bcrypt.hash(password, 12);
	// Result: $2b$12$R9h/cIPz0gi.URNNXjiYAu... ← salt is inside this string
	Store only passwordHash in DB — salt is part of it.
	
	On login:
	
	await bcrypt.compare(enteredPassword, storedHash); // extracts salt

Does bycrypt uniformly apply the authorization ? is it natively compatible with claude code?	bcrypt only hashes and verifies passwords (part of authentication). It does not handle authorization (who can do what). After login, you still check roles/permissions in middleware or API routes separately. It's just an npm package (bcryptjs / bcrypt) — works in any Node.js/Next.js project, including one you build with Claude Code. No special integration; you install it and import it.

How refresh token works with NextAuth.Js? Please give code example.	NextAuth session can auto-refresh on activity (updateAge). For OAuth provider tokens (Google, Azure), you store the provider's refresh token in the jwt callback and refresh when the access token expires: // auth.ts
	import NextAuth from "next-auth";
	import GoogleProvider from "next-auth/providers/google";
	
	async function refreshAccessToken(token) {
	const res = await fetch("https://oauth2.googleapis.com/token", {
	method: "POST",
	headers: { "Content-Type": "application/x-www-form-urlencoded" },
	body: new URLSearchParams({
	client_id: process.env.GOOGLE_CLIENT_ID!,
	client_secret: process.env.GOOGLE_CLIENT_SECRET!,
	grant_type: "refresh_token",
	refresh_token: token.refreshToken as string,
	}),
	});
	const data = await res.json();
	if (!res.ok) throw data;
	return {
	...token,
	accessToken: data.access_token,
	expiresAt: Math.floor(Date.now() / 1000) + data.expires_in,
	refreshToken: data.refresh_token ?? token.refreshToken,
	};
	}
	
	export const authOptions = {

How refresh token works with NextAuth.Js? Please give code example.	export const authOptions = {
	providers: [GoogleProvider({ clientId: "...", clientSecret: "..." })],
	session: { strategy: "jwt", maxAge: 30 * 24 * 60 * 60 }, // 30 days
	callbacks: {
	async jwt({ token, account }) {
	if (account) {
	token.accessToken = account.access_token;
	token.refreshToken = account.refresh_token;
	token.expiresAt = account.expires_at;
	return token;
	}
	if (Date.now() < (token.expiresAt as number) * 1000) return token;
	return refreshAccessToken(token);
	},
	async session({ session, token }) {
	session.accessToken = token.accessToken;
	return session;
	},
	},
	};

does redux and zustand server the same purpose?	Yes — same purpose: manage client-side state in React. Different style and complexity. For PawMatch: view toggle (grid/list) and UI state → Zustand is enough. Pet list from API → server state (React Query/SWR), not Redux/Zustand.
Which caching technique is used by TanStack query?	Stale-While-Revalidate (SWR) — the core pattern. How it works: Return cached data immediately (even if stale) Refetch in background from API Update UI when fresh data arrives
Where does useQuey save cache? like in indexDB on client side or on server-side?	Client-side only — in browser memory (RAM) by default. useQuery cache lives in the JavaScript heap on the client while the app is open. Not on the server, not in IndexedDB by default
with Tanstack, do we have an api call from UI to provide the data? or?	Yes — you still need an API call. TanStack Query doesn't provide data by itself.

the provider of tanstack is similar to context api used in reat , does it use the same principle ??	Similar mechanism, different purpose. Both use React Context under the hood — a Provider wraps your app and child components access shared data. // Same pattern — Provider wraps app <QueryClientProvider client={queryClient}> <App /> </QueryClientProvider>
Which caching technique is used by TanStack query?	Stale-While-Revalidate (SWR) — the core pattern. How it works: Return cached data immediately (even if stale) Refetch in background from API Update UI when fresh data arrives
SO if user refresh page then TanStack query cache will be gone?	Yes — by default the cache is gone on page refresh. It's stored in browser memory (RAM), which clears when the page reloads. TanStack Query will fetch from the API again on refresh. To survive refresh: use the persist plugin to save cache to localStorage or IndexedDB — but that's optional and not the default.
In a react app, how will the data leak to other users. This is supposed to be client side right?	Client-side data in one browser cannot leak to another user's browser. Each user has their own isolated browser memory, cookies, and cache. TanStack Query cache — only in that user's browser Zustand/Redux state — same, per browser session View toggle, UI prefs — no leak risk

In that case, why is useState necessary here?	TanStack Query handles server data — useState handles everything else local to the component. Use useState for:
	
	Form inputs (name, email while typing)
	Modal open/close
	Selected tab, dropdown value
	Temporary UI state before submit
	Use TanStack Query for:
	
	Data from API (pet list, user profile)
	Cached, refetched server state Example in PawMatch: const { data: pets } = useQuery({ queryKey: ["pets"], queryFn: fetchPets }); // server
	const [isModalOpen, setIsModalOpen] = useState(false); // local UI
	const [searchText, setSearchText] = useState(""); // local filter input

so useState is also for client side state, when to use redux vs useState vs zustand?	useState State in one component — form input, modal open, toggle Zustand State shared across many components — view mode (grid/list), theme, filters used in multiple pages Redux Large app, many teams, strict patterns — complex global state, time-travel debugging, middleware

Can TanStack Query be used with Mongoose, GraphQL, or Prisma?	Yes — all of them. TanStack Query doesn't care what's behind the API. // Prisma via API route
	useQuery({ queryKey: ["pets"], queryFn: () => fetch("/api/pets").then(r => r.json()) });
	
	// GraphQL
	useQuery({ queryKey: ["pets"], queryFn: () => graphqlClient.request(GET_PETS) });
	
	// Mongoose — same pattern, data comes through your API
	useQuery({ queryKey: ["pets"], queryFn: () => fetch("/api/pets").then(r => r.json()) });

what if we do update and delete on the same view? how many collection we need?	TanStack Query (cache): Usually one queryKey for the list, e.g. ["pets"]. Update/delete use useMutation, then invalidate the same key to refresh the view
	const { data } = useQuery({ queryKey: ["pets"], queryFn: fetchPets });
	
	const deletePet = useMutation({
	mutationFn: (id) => fetch(`/api/pets/${id}`, { method: "DELETE" }),
	onSuccess: () => queryClient.invalidateQueries({ queryKey: ["pets"] }),
	}); No separate cache collection per action — same key, refetch after mutate.

useQuery uses cache which is shared across all the users as well?	No. useQuery cache is only in that user's browser — not shared across users. Each user gets their own isolated cache in their own browser memory. User A's cache never reaches User B.

can we have component with both view and update?	function PetCard({ pet }) {
	const queryClient = useQueryClient();
	
	const updatePet = useMutation({
	mutationFn: (data) => fetch(`/api/pets/${pet.id}`, { method: "PUT", body: JSON.stringify(data) }),
	onSuccess: () => queryClient.invalidateQueries({ queryKey: ["pets"] }),
	});
	
	return (
	
	
	 {pet.name} — {pet.age} {/* view */}
	<button onClick={() => updatePet.mutate({ adopted: true })}>
	Apply to Adopt {/* update */}
	</button>
	
	);
	}

will invalidating cache makes system slow as we have to fetch data from server again?	Yes, it triggers a refetch — but that's intentional and usually fine. What happens on invalidate: Cache marked stale → background refetch from server → UI updates with fresh data

What is the purpose of useQuery then? I can directly fetch data from server and render it inside useState? Because useQuery is not shared across all users, if someone else added a puppy, it wont displayed to other users until they refresh the screen	Yes, you can use useState + fetch — it works. useQuery doesn't magically share data across users. useQuery = better developer experience + smart refetching for one user. Cross-user live updates need polling, websockets, or refetch strategies — regardless of useState or useQuery. For PawMatch, refetch on focus + invalidate after your own actions is usually enough.
But this can conflicting in multi user applications ? Optimistic Update	Yes — optimistic updates can conflict in multi-user apps. The problem: User A optimistically marks puppy as adopted → UI shows adopted User B also adopts same puppy at the same time Both think they succeeded — but only one can actually adopt For PawMatch: Heart/favorite → optimistic update is fine (low conflict) Apply to adopt → pessimistic (wait for server) — puppy can only go to one person
will these changes update the UI of pther users? synchronize?	No — not automatically. Other users' UIs only update when their browser refetches.
is this a key value store globally or specific to user?	pecific to each user/browser — not global. TanStack Query cache is a key-value store in that user's browser memory: ["pets"] → [Biscuit, Nimbus, Pepper] ["favorites"] → [1, 3]

can we have different saletime of every query?	es — every query can have its own staleTime.
	
	useQuery({ queryKey: ["pets"], queryFn: fetchPets, staleTime: 60_000 }); // 1 min
	useQuery({ queryKey: ["profile"], queryFn: fetchProfile, staleTime: 5 * 60_000 }); // 5 min
	useQuery({ queryKey: ["stats"], queryFn: fetchStats, staleTime: 0 }); // always stale
	Or set a default globally, override per query:
	
	const queryClient = new QueryClient({
	defaultOptions: { queries: { staleTime: 30_000 } },
	});

can we use the tanstack query in the server side components ?	Server Components fetch with fetch/Prisma. TanStack Query runs on the client — or prefetch on server and hydrate to client.
where does Tanquery store the data? In client side or server side?	Client side only — in the browser's memory (RAM) by default. Not on the server. Each user has their own separate cache in their browser. Optional: persist to localStorage or IndexedDB with a plugin — still on that user's device, not shared.
this may create memory issues for a large data. rt?	Yes — if you cache too much large data in browser memory, it can cause issues. Use pagination + sensible gcTime — TanStack Query is meant for reasonable API response sizes, not dumping entire DB into browser RAM.
So if we are using mostly the server side components in next or react we can use fetch and when we need to call any api's from client browser we need to use tanstack ?	Yes — that's the right mental model. Where How to fetch Server Component fetch() or Prisma directly — runs on server, no TanStack needed Client Component TanStack Query (useQuery) — caching, loading states, refetch
do we have any cache size limits for the tanstack and zustand also stores the data in the RAM ?	No hard cache size limit by default — grows until gcTime removes unused entries You control it via gcTime, pagination, and how much data you fetch Browser RAM is the real limit — too much data = slow/crash
Whats primary key?	A primary key is the unique ID for each row in a table — no two rows can share it.
what are the best practices? any documentation available?	TanStack Query tanstack.com/query/latest/docs TanStack best practices Query guides Zustand docs.pmnd.rs/zustand Next.js data fetching nextjs.org/docs/app/building-your-application/data-fetching NextAuth authjs.dev Prisma prisma.io/docs

on migrations, is it difficult to revert, whats flyway, what do we have similar for tan stack	Reverting migrations — is it hard? Can be tricky. Depends on the change: Easy: add column → revert = drop column Hard: drop column with data, rename, merge tables — data may be lost Most tools don't auto-revert — you write a down migration manually or restore from backup. Test migrations on staging firs

How the broadcasting work in client side ?	1. Across users (real-time sync) — WebSockets / SSE
	
	Server pushes updates when data changes; all connected browsers receive it.
	
	User A adopts puppy → API saves to DB → server broadcasts event
	→ User B's browser receives → TanStack Query invalidates → UI updates
	Tools: Socket.io, Pusher, Ably, Supabase Realtime, or native WebSockets.
	
	// Client listens
	socket.on("pet-adopted", () => {
	queryClient.invalidateQueries({ queryKey: ["pets"] });
	});
	2. Across tabs (same user, same browser) — BroadcastChannel API
	
	One tab updates → other tabs hear it. No server needed.
	
	const channel = new BroadcastChannel("pawmatch");
	channel.postMessage({ type: "favorite-changed", petId: 1 });
	channel.onmessage = (e) => queryClient.invalidateQueries({ queryKey: ["pets"] });
	3. TanStack Query — not true broadcasting
	
	invalidateQueries only affects that one browser. Other users need WebSockets/polling.

By AI, any specific ai tool ?	Depends what you're using AI for — no single mandatory tool. Schema / DB design ChatGPT / Claude — draft tables, review relations, generate Prisma schema from description Prisma + AI — describe models in plain English, paste into schema.prisma

any live project eg for db + AI , benfits seen	DB + AI = Benefits: speed, personalization,
What is difference of hashed with salt and encryption?	Hash + salt = one-way, for passwords. Encryption = two-way, for data you need to retrieve later.
I can see only a word file not any module_4_lab folder	Dear Learners, The code files and pre-class installation checklist for today's hands-on session have been uploaded on the Prism Platform. Kindly refer to the same and complete the required setup before joining today's hands-on session, which will be conducted after the Professor's lecture. Access Path : Prism Platform → My Programs → EPGC AI Native Software Engineering → Live Learning → Upcoming Sessions → Session Name → View Details → Resources
please paste steps for postgress installation	macOS (Homebrew — most common) # 1. Install brew install postgresql@16 # 2. Start service brew services start postgresql@16 # 3. Verify psql --version # 4. Open Postgres shell psql postgres # 5. Create DB (inside psql) CREATE DATABASE pawmatch; \q

I checked in my ubuntu terminal (my laptop is Windows). THere is psql installed. Does this mean its installed on windows aswell? I tried it in terminal but its not able to find psql when running psql --version	No — WSL Ubuntu and Windows are separate environments. psql in Ubuntu terminal = installed inside Linux (WSL) only. Windows CMD/PowerShell doesn't see it. Option A — use WSL (easiest, you already have it): # In Ubuntu terminal psql --version psql postgres Run your app from WSL too, or point DATABASE_URL to localhost:5432.
i have missed live answer for : are there any study materials prescribed for full understanding and covering full syllabus, this questions for all the sessions , sepcially for session like fondation models where topic is vast	Next.js data fetching nextjs.org/docs/app/building-your-application/data-fetching TanStack Query tanstack.com/query/latest/docs Zustand docs.pmnd.rs/zustand
This Front end also we have to develope--the local UI that Soham sir is showing?	Yes